TER Project: SHACL Shapes Generator and Validator

Salma Alkhalily
Supervised by: Dr. Fatiha Sais

February 2025

Abstract

Generating SHACL shapes is crucial for validating that data graphs satisfy a set of conditions. SHACL shapes can also be viewed as a description of the data graphs that meet these conditions. Such descriptions have various uses beyond validation, including the creation of user interfaces, the generation of code, and data integration.

The validation process produces a report that contains violations and describes the conditions that were violated.

This project develops a tool to generate keys for RDF data sources and uses these keys to create SHACL shapes for validation purposes. The generated SHACL shapes are validated against the original data source or other data sources that share the same keys. Validation reports are generated as a result of the validation process, including detailed explanations of any violations.

In this project, two scenarios are considered. In the first scenario, SHACL shapes are generated directly from the keys and validated against the RDF data source. In the second scenario, a mapping file is used that contains translated properties corresponding to those used in generating keys. SHACL shapes are then created for these translated properties and validated against the data source. This approach enables validation across multiple data sources that share the same property structures but have different property names.

The workflows were implemented using Apache Jena, a robust semantic data processing framework, and a user-friendly interface was developed with JavaFX and minimal CSS. This report provides a detailed explanation of each scenario, their methodologies, and results, supported by examples and illustrations.

1 Introduction

1.1 Background

The Resource Description Framework (RDF)[1] is a standard model for data interchange on the Web. It is designed to facilitate data merging, even when the underlying schemas differ, and to support the evolution of schemas over time without requiring changes from all data consumers. RDF uses Uniform Resource Identifiers (URIs) to name relationships between resources, forming "triples" (subject-predicate-object). This simple, flexible model enables structured and semi-structured data to be mixed, exposed, and shared across different applications.

RDF graphs are essentially directed, labeled graphs where edges represent the named links between resources (represented as graph nodes). This graph structure provides an intuitive mental model for RDF and is commonly used in visual explanations of the framework.

The Shapes Constraint Language (SHACL)[2] is a powerful language used to validate RDF graphs against a set of conditions. These conditions are specified in the form of shapes, which are also RDF graphs.

The validation[3] process involves checking a data graph (the RDF data being validated) against a shapes graph (the set of conditions defined in SHACL). The results of the validation indicate whether the data graph conforms to the shapes and highlight any violations.

As the complexity of RDF datasets increases, particularly when integrating data from multiple sources, maintaining data integrity becomes more challenging. SHACL provides a means to enforce consistency and ensure that RDF data adheres to predefined schemas and structures.

1.2 Problem Statement

With the growing volume of RDF data, it is crucial to validate data across multiple sources, especially when those sources share similar structures but use different property names. However, generating SHACL shapes for keys of such data and ensuring that they are validated consistently can be a complex task.

This project aims to address this challenge by developing a tool that generates SHACL shapes based on keys derived from RDF data sources. The tool will validate the generated shapes against both the original data source and other data sources that share similar structures.

1.3 Objectives of the Study

The main objectives of this project are:

[1] https://www.w3.org/RDF/
[2] https://www.w3.org/TR/shacl/
[3] https://www.w3.org/TR/shacl/

- To develop a tool for generating SHACL shapes from keys derived from the properties of RDF data sources.
- To validate the generated SHACL shapes against RDF data sources.
- To compare two scenarios of SHACL shape generation:
  - **Scenario 1:** SHACL shapes are generated directly from keys derived from the properties of an RDF data source without any translations.
  - **Scenario 2:** A mapping file is used to translate the property names before generating and validating the SHACL shapes, allowing for validation across different data sources that have a similar property structure but different property names.
- To evaluate the effectiveness of these scenarios in generating valid SHACL shapes and ensuring data integrity.

1.4 Methodology

The workflows for SHACL shape generation and validation were implemented using Apache Jena, a robust semantic web framework. Jena was chosen for its powerful capabilities in processing RDF data and integrating with SHACL validation. Additionally, a user-friendly interface was developed using JavaFX and minimal CSS to allow users to interact with the tool easily.

1.5 Structure of the Report

This report is structured as follows:

- Approach explains the approach for generating keys, creating SHACL shapes for RDF data sources, and validating the generated shapes against RDF data sources, including the two scenarios.
- Implementation presents the implementation details, including the technologies used and how the tool was developed.
- Evaluation shows different successful and failed cases and discusses the results of the validation process and the generated SHACL shapes.
- Conclusion and Recommendations concludes the report and suggests areas for future work.
- References.

2 Approach

This chapter describes the approach for generating SHACL shapes and validating them against RDF data sources. Two scenarios are implemented:

- **Without translated properties**: SHACL shapes are generated using the original property names.
- **With translated properties**: A mapping file is used to translate property names before generating SHACL shapes, allowing validation across different data sources with similar structures but different property names.

The implementation consists of three main parts:

1. **Generating keys** (common to both scenarios).
2. **Generating SHACL shapes** (varies by scenario).
3. **Validating RDF data against SHACL shapes**.

Each part is described in detail in the following sections.

2.1 Generating Keys Process

This process is performed using the **SAKey tool** and is shared between both scenarios.

2.1.1 Process Overview



2.1.2 Key Definitions

- **Key**: A set of properties that uniquely identifies RDF resources.
- **Almost key**: A set of properties that is not a key due to a few exceptions (i.e., some resources do not follow the key constraint).
- **n-almost key**: A set of properties is an *n-almost key* if at most n instances share the same values for that set of properties.

2.1.3 SAKey Tool

SAKey (Scalable Almost Key Discovery) is an approach for discovering keys in RDF data efficiently. It dynamically detects data characteristics to reduce the search space and can discover keys in datasets with erroneous data or duplicates (i.e., almost keys).

SAKey Input Parameters

- **File path**: The RDF dataset file where key discovery takes place. SAKey assumes this file contains instances of a single class.
- **Number of exceptions (n)**: Defines how many exceptions are allowed in discovering almost keys.

Command to run SAKey:

```
java -jar "jar_fullpath"/SAKey.jar "file_fullpath" n
```

Example:

```
java -jar /Users/salma/SAKey.jar /Users/salma/datasets/DB_Lake.nt 1
```

2.2 Generating SHACL Shapes for Keys

The generated SHACL shapes enforce **unique constraints** using SPARQL[4] constraints and rules to validate RDF data.

2.2.1 SHACL Shape Generation Process

For each **almost key**, a SHACL shape is defined, including:

1. **Property constraints**:
   - Each property in the key must be a string.
   - Each property can have at most one value per subject.
   - The shape specifies the property being validated.
2. **Duplication constraint**: Ensures that no two nodes share the same value for an almost key.

2.2.2 Example Scenarios

Case 1: Violation Detected Composed key k_1 consists of two properties p_1 and p_2:

```
p1(D1, z1) and p2(D1, w1)
p1(D2, z1) and p2(D2, w1)
```

Since $D1 = D2$, a **violation is triggered** (duplicate detected).

Case 2: No Violation Composed key k_2 consists of two properties p_1 and p_2:

```
p1(D1, z1) and p2(D1, w1)
p1(D2, z2) and p2(D2, w2)
```

Since $D1 \neq D2$, **no violation is detected**.

[4]SPARQL Query Language for RDF

2.2.3 SHACL Triple Rule

If two resources share the same value for an almost key, they should be marked as `owl:sameAs`:

```
sameAs(D1, D2)
```

If this condition is not met, a **violation** is triggered.

2.2.4 Differences Between the Two Scenarios

- **Without translated properties**: Generate SHACL shapes using the original property names.
- **With translated properties**: Use a **mapping file** to translate property names before generating SHACL shapes.

2.3 Validating RDF Data Against SHACL Shapes

This step involves validating the RDF data source using the generated SHACL shapes and producing a **validation report**.

2.3.1 Validation Process

1. **Preprocessing the RDF Data Source**
   - Remove empty lines.
   - Split RDF triples into **subject** — **predicate** — **object**.
   - Transform URIs and object literals.
   - Replace certain symbols.
   - Perform preprocessing using **Apache Spark**.
2. **Parse SHACL Shapes Constraints**
3. **Perform SHACL Validation**
4. **Generate Validation Report**
   - Count the **total number of violations**.
   - Count the **number of violations per node**.